Software Engineering Tools for Quantum Computing: Systematic Mapping Study

Camila Ferreira Alves
Universidade Federal Fluminense
Niterói, Brasil
cf_alves@id.uff.br

Leonardo Murta
Universidade Federal Fluminense
Niterói, Brasil
leomurta@ic.uff.br

Luis Kowada
Universidade Federal Fluminense
Niterói, Brasil
luis@ic.uff.br

Abstract

Quantum Computing (QC) is rapidly evolving from a theoretical paradigm into a practical computing platform, increasing the demand for Software Engineering (SE) tools to support the development of reliable and maintainable quantum software. Although numerous SE tools have recently been proposed, the current landscape remains fragmented and lacks a comprehensive synthesis. This paper presents a Systematic Mapping Study (SMS), aiming to identify the categories of SE tools proposed for QC, analyze the empirical evidence supporting these tools, and summarize the main limitations, challenges, and research gaps reported in the literature. Following established guidelines, a total of 4,489 publications were identified using database searches complemented by backward and forward snowballing, and 71 primary studies were selected after applying the predefined inclusion and exclusion criteria. The results reveal eight SE tool categories spanning multiple phases of the QC software development lifecycle. However, the literature is predominantly concentrated on software implementation and testing, while requirements engineering, software design, maintenance, and reuse remain comparatively underexplored. Controlled experiments constitute the dominant evaluation method, whereas industrial case studies and practitioner-oriented evaluations are scarce. Furthermore, recurring challenges include scalability limitations, Noisy Intermediate-Scale Quantum (NISQ) hardware constraints, the oracle problem, limited interoperability, and the rapid evolution of QC software ecosystems. These findings provide a comprehensive overview of the current state of SE for QC and highlight opportunities for future research toward integrated, scalable, and empirically validated development toolchains.

Keywords

Quantum Computing, Quantum Software Engineering, Software Engineering Tools, Quantum Software Development Lifecycle

1 Introduction

QC is a computational paradigm based on the principles of quantum mechanics, using quantum bits (qubits) as its fundamental unit of information [22]. In contrast to classical bits, which can assume only one of two possible binary states, qubits can exist in a superposition of states and exhibit quantum correlations through entanglement, enabling forms of information processing that are not possible in classical computing [18]. These properties, combined with quantum interference, enable the development of algorithms that can provide computational advantages for specific classes of problems, including integer factorization, database search, optimization, and the simulation of quantum systems [8, 19, 36].

In this context, Quantum Software Engineering (QSE) has emerged as a research area dedicated to the adaptation and development of methods, processes, techniques, and tools to support all phases of the quantum software lifecycle [30]. As in classical Software Engineering, the availability of specialized tools plays a fundamental role in improving developer productivity, enhancing software quality, and making software development processes more systematic and reproducible.

In recent years, numerous tools have been proposed to support SE activities in QC [30]. These solutions include software development kits (SDKs), development frameworks, integrated development environments (IDEs), simulators, testing and debugging tools, verification and validation tools, modeling environments, and workflow management and orchestration mechanisms for hybrid classical–quantum applications. However, these proposals are scattered throughout the literature and often focus on specific stages of the software lifecycle, making it difficult to obtain a comprehensive view of the current state of the art [29].

Furthermore, significant gaps remain regarding the maturity of these tools, the extent to which they support SE activities, the empirical evidence used to validate their effectiveness, and the challenges and limitations reported by their authors [30]. The lack of a structured synthesis hinders the identification of research trends and opportunities, as well as the development of more comprehensive solutions for the field.

Motivated by these challenges, this study presents a SMS aimed at identifying, cataloging, and characterizing tools designed to support SE activities in the development of quantum software systems. Specifically, the SMS seeks to answer research questions about the categories of tools proposed to support software lifecycle activities, the empirical evidence used to evaluate these tools, and the limitations, challenges, and research gaps that remain in the current literature. To ensure methodological rigor, the SMS follows established guidelines for secondary studies in SE and adopts a snowball strategy to identify relevant primary studies [12]. The findings are expected to provide a consolidated view of the current state of the art, assisting both researchers and practitioners in understanding the existing ecosystem of QSE tools while highlighting opportunities for future research.

2 Research Method

This SMS follows the guidelines for conducting Mapping Studies (MS) in Software Engineering. To ensure the coverage of relevant studies through the protocol while maintaining a systematic, transparent, and reproducible selection process, the proposed protocol combines the construction of an initial study set (*Start Set*) with an iterative snowballing strategy [44].

2.1 Research Questions

The research questions were defined based on the objective of providing a comprehensive overview of the current landscape of SE tools for QC. Specifically, they aim to identify the types of tools proposed in the literature, understand how these tools have been evaluated, and uncover the main limitations and research opportunities reported in existing studies.

Accordingly, the following research questions guide this Mapping Study:

- **RQ1:** *What categories of SE tools have been proposed for QC, and which phases of the Quantum Software Development Lifecycle (QSDL) do they support?*
  To address this question, we adopt the QSDL proposed by Dwivedi et al. [6] as our conceptual framework. This lifecycle adapts traditional SE to the quantum domain across six phases: (i) Requirements Analysis (QSRA), (ii) Design, (iii) Implementation, (iv) Testing, (v) Maintenance, and (vi) Reuse. We map each identified tool to its corresponding QSDL phase(s) based on the specific development activities it supports, establishing a structured classification across the QSE pipeline.
- **RQ2:** *What types of empirical evidence have been used to evaluate these tools?*
  This research question investigates the empirical evaluation approaches adopted in the literature for assessing SE tools for QC. To classify the evaluation methods, we adapted the empirical study taxonomy proposed by Zannier et al. [47]. The resulting classification, summarized in Table 1, includes Controlled Experiment, Case Study, Example Application, Survey, Discussion, Performance Evaluation, and Proof of Concept (PoC), covering a spectrum from rigorous empirical studies to preliminary feasibility validations.
- **RQ3:** *What limitations, challenges, and research gaps have been reported in the existing literature?*
  This research question focuses on identifying and synthesizing the main limitations, challenges, and research gaps reported in studies proposing or evaluating SE tools for QC. The goal is to highlight recurring issues such as limited empirical validation, scalability constraints, tool maturity, and incomplete coverage of the QSDL phases, thereby outlining opportunities for future research.

2.2 Start Set Construction

The *Start Set* was established through a manual inspection of papers published in the *Workshop on QSE* [1]. Titles, abstracts, and keywords were analyzed to identify studies proposing, evaluating, or discussing SE tools, frameworks, SDKs, development environments, testing and debugging tools, modeling approaches, simulators, or workflow orchestration mechanisms for quantum applications.

2.3 Study Selection

The study selection process consisted of three sequential screening stages.

[1] https://dblp.org/db/conf/icse-qse/index.html

Initially, the titles and abstracts of the retrieved studies were analyzed to eliminate clearly irrelevant publications. Subsequently, the introduction of each remaining paper was examined to verify its alignment with the objectives of this SMS. Finally, the full text of the candidate studies was read, and the inclusion and exclusion criteria were applied.

The following inclusion criterion was adopted:

- **IC1:** The study presents, proposes, evaluates, or discusses a SE tool or framework for QC.

The following exclusion criteria were applied:

- **EC1:** Studies not written in English.
- **EC2:** Studies that merely use existing tools without presenting a SE contribution.
- **EC3:** Tutorials, posters, keynote papers, or extended abstracts lacking sufficient technical content.
- **EC4:** Duplicate publications.
- **EC5:** Studies that do not provide a minimum architectural or functional description of the proposed tool.
- **EC6:** Studies that are not freely accessible.
- **EC7:** Studies that are not peer-reviewed.
- **EC8:** Studies that do not present, propose, evaluate, or discuss a SE tool or framework for QC.
- **EC9:** Studies presenting only theoretical models or mathematical formalisms without an implemented artifact.
- **EC10:** Secondary studies (e.g., surveys or literature reviews) that do not propose or evaluate an original artifact.
- **EC11:** Studies whose primary contribution is unrelated to the proposed tool itself.

Table 1: Types of empirical evaluation used in the analysis

Type	Description
Controlled Experiment	Experimental study design in which independent variables are manipulated and control conditions are established to investigate causal relationships between factors and outcomes [3].
Case Study	Empirical investigation in a real-world context [46].
Example Application	Demonstration using a representative example [34].
Survey	Collection of participants' opinions or experiences [26].
Discussion	Qualitative analysis without formal empirical evaluation [47].
Performance Evaluation	Systematic assessment of a tool's performance, feasibility, or effectiveness using quantitative metrics such as efficiency, accuracy, scalability, or resource consumption under controlled or semi-controlled settings [43].
Proof of Concept	Preliminary validation of technical feasibility [13].

2.4 Snowballing Procedure

After the definition of the *Start Set*, an iterative snowballing procedure was performed following the guidelines proposed by Wohlin [44]. To support the management and execution of the snowballing process, the Parsif.al platform[2] was used.

Backward snowballing consisted of examining the reference lists of all selected studies to identify additional relevant publications. Forward snowballing was then conducted using the Google Scholar[3] *Cited by* feature to identify studies citing the selected papers.

All candidate studies identified during these stages were subjected to the same screening procedure and inclusion and exclusion criteria. Studies meeting the eligibility criteria were incorporated into the selected set and became the basis for subsequent snowballing iterations.

The process continued until saturation was reached, that is, until no new relevant studies were identified.

2.5 Data Extraction

After the final set of primary studies was established, data were systematically extracted using a standardized electronic form developed for this study.

The extraction form was designed to ensure consistency and traceability throughout the data collection process. For each selected study, the following information was recorded: paper title, authors, tool name and purpose, source study, related studies (when applicable), repository availability, supported quantum platforms, tool category, supported SE activities, software lifecycle phase, empirical validation, and the limitations, challenges, and research gaps reported by the authors.

The extracted data were subsequently analyzed and synthesized to answer the research questions, characterize the current landscape of SE tools for QC, identify research trends, and highlight opportunities for future work.

3 Results

The study selection process resulted in the assessment of a total of 4,489 publications, including the 12 articles comprising the start set, with five iterations of backward and forward snowballing subsequently performed. In the first stage of the selection process, EC8 was applied to identify and exclude studies that did not meet the predefined criteria, resulting in the exclusion of 1,712 studies. The remaining studies were then assessed according to the other predefined exclusion criteria as follows: EC1 (19 studies), EC2 (36 studies), EC3 (28 studies), EC4 (2,171 studies), EC5 (12 studies), EC6 (10 studies), EC7 (96 studies), EC9 (145 studies), EC10 (124 studies), and EC11 (53 studies). After applying the inclusion and exclusion criteria, 83 studies remained. Subsequently, 13 studies reporting the same research were grouped to avoid duplicate evidence, resulting in a final set of 71 primary studies for data extraction and analysis.

[2]https://parsif.al/
[3]https://scholar.google.com/

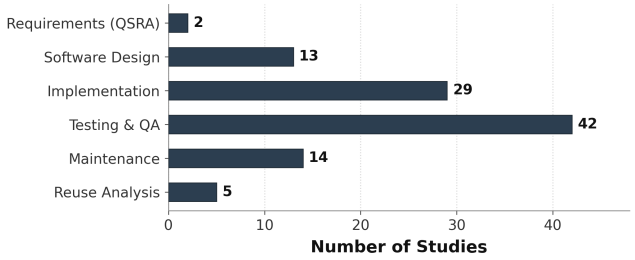


Figure 1: Distribution of the selected primary studies according to the QSDL phases they support. A single study may contribute to multiple lifecycle phases.

3.1 What Categories of SE Tools Have Been Proposed for QC, and Which Phases of the QSDL Do They Support?

The analysis of the selected studies identified a broad ecosystem of SE tools supporting QC. These tools were classified into eight major categories according to their primary purpose:

- **Requirements and Modeling Tools**, supporting the specification and conceptual modeling of quantum and hybrid classical–quantum systems [10, 20, 27].
- **Code Generation and Model-Driven Engineering Tools**, which automatically transform models into executable quantum code using model-driven approaches [9, 11, 32].
- **Programming Assistants and AI-based Coding Tools**, providing intelligent support during software implementation through conversational interfaces, code completion, and code generation techniques [2, 4, 5].
- **Static Analysis and Quality Assurance Tools**, aimed at detecting programming errors, API misuse, code smells, and quality issues without executing the program [24, 35, 48].
- **Testing Frameworks**, including property-based, mutation, metamorphic, concolic, regression, and diversity-guided testing approaches [1, 23, 31].
- **Runtime Verification and Assertion Frameworks**, responsible for monitoring quantum programs during execution through assertions and runtime specifications [17, 45].
- **Debugging and Program Understanding Tools**, assisting developers in fault localization, visualization, and software comprehension [14, 25, 33].
- **Execution, Middleware, and Maintenance Tools**, supporting interoperability, execution management, software modernization, optimization, and configuration management [21, 28, 42].

Although tools were identified across multiple SE activities, their distribution throughout the QSDL is far from uniform.

Figure 1 presents the distribution of the selected studies according to the lifecycle phases they support. Since many tools provide functionalities that span multiple activities, a single primary study may contribute to more than one QSDL phase. Consequently, the total number of occurrences shown in the figure exceeds the total number of selected studies.

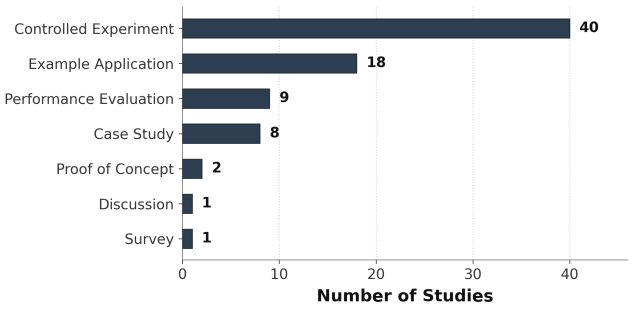


Figure 2: Distribution of empirical evidence used to evaluate QSE tools. A single study may employ multiple evaluation approaches.

The results indicate a strong concentration of research efforts on the *Testing* phase, followed by *Implementation*. This finding reflects the current maturity of QSE, where the primary concern is ensuring the correctness, reliability, and quality of quantum software through testing techniques, debugging approaches, static analyses, runtime assertions, and verification mechanisms. Most recently proposed tools aim to detect programming errors, validate quantum program behavior, improve fault localization, or automate quality assurance activities.

A second group of studies focuses on the *Design* and *Maintenance* phases. These works mainly propose model-driven engineering techniques, UML extensions, automatic code generation, software modernization approaches, middleware solutions, and interoperability mechanisms to facilitate the development and evolution of hybrid classical–quantum software systems.

In contrast, *Software Reuse* is supported by relatively few studies. Existing approaches mainly focus on reusable modeling artifacts, runtime specifications, design patterns, and modular verification techniques. The limited number of contributions suggests that reuse of quantum software assets remains an open research challenge.

Overall, the results reveal that current research in QSE is predominantly quality-oriented, with significant emphasis on testing, debugging, verification, and implementation support. Earlier lifecycle activities, such as requirements engineering, receive comparatively little attention, while reuse and long-term software evolution are still emerging research topics. These findings indicate opportunities for future work toward a more balanced tool ecosystem capable of supporting the entire Quantum Software Development Lifecycle.

3.2 What Types of Empirical Evidence Have Been Used to Evaluate These Tools?

To investigate how SE tools for QC have been empirically evaluated, this study analyzed the evaluation approaches reported in the selected primary studies. The identified approaches were classified according to the type of empirical evidence provided, covering a spectrum from preliminary feasibility assessments to more rigorous experimental evaluations. Since a single study may employ multiple evaluation approaches, it may contribute to more than one evidence category.

Figure 2 presents the distribution of the empirical evidence identified in the selected studies. The results indicate that *Controlled Experiments* are the most frequently adopted evaluation approach, reported in 40 studies. This prevalence suggests that many proposed tools are assessed through structured experimental settings, where predefined tasks, benchmark algorithms, and quantitative metrics are employed to measure their effectiveness, performance, or applicability.

The second most common evaluation method is *Example Application*, reported in 18 studies. Rather than performing rigorous comparative analyses, these works demonstrate the applicability of the proposed tools by applying them to representative quantum algorithms or realistic software development scenarios.

Performance Evaluation appears in nine studies, while *Case Studies* are reported in eight studies. These approaches generally aim to evaluate the feasibility, effectiveness, or performance of the proposed tools under specific experimental settings or practical use cases.

Only a small number of studies employ other forms of empirical evidence. Two studies present a *Proof of Concept*, whereas only one study each reports an *Empirical Study*, a *Discussion*, or a *Survey*. The scarcity of surveys and empirical studies involving practitioners suggests that user-centered evaluation and industrial validation remain largely unexplored in QSE.

Overall, the results reveal that the empirical validation of QSE tools is predominantly technology-oriented. The literature focuses on demonstrating the technical feasibility and effectiveness of proposed solutions through controlled experiments, while relatively little attention has been devoted to evaluating usability, developer experience, adoption in practice, or industrial applicability. These findings highlight an important opportunity for future research to complement technical validation with practitioner-centered evaluations and real-world industrial studies.

3.3 What Limitations, Challenges, and Research Gaps Have Been Reported in the Existing Literature?

The analysis of the selected studies reveals that QSE still faces several structural challenges that limit the maturity and practical adoption of existing tools. Figure 3 summarizes the main limitations, challenges, and research gaps reported in the literature, indicating the frequency with which each issue was identified across the selected studies. The results show that scalability and simulation constraints, lifecycle-related challenges, and hardware limitations are among the most frequently reported issues, suggesting that current QSE approaches still struggle to support realistic quantum software development scenarios.

The most frequently reported limitation concerns scalability and simulation, as illustrated in Figure 3. Many existing SE techniques rely on classical simulation, whose computational cost increases exponentially with the number of qubits. As a consequence, testing, debugging, and verification approaches are commonly evaluated using relatively small benchmark circuits, often containing fewer than 30 qubits. Similar scalability issues have also been observed in symbolic simulation and symbolic execution techniques, which suffer from state-space explosion when analyzing complex quantum

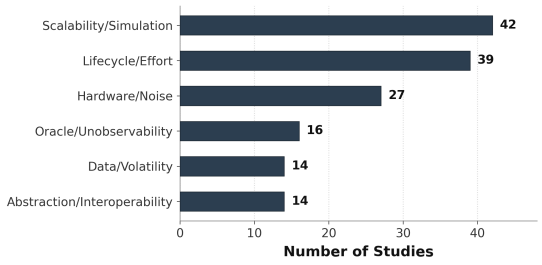


Figure 3: Frequency of limitations, challenges, and research gaps reported across the selected studies.

programs. These limitations reduce the ability of current tools to evaluate realistic quantum applications and represent a significant barrier to their practical adoption [7, 31, 37, 40, 41].

Another major challenge identified in the literature concerns the complete software lifecycle and the effort required to develop and maintain quantum software. Although several tools have been proposed for specific activities, most approaches remain focused on isolated development tasks rather than providing integrated support throughout the quantum software lifecycle. The rapid evolution of quantum programming languages, software development kits, and cloud-based quantum platforms requires continuous adaptation of development tools, causing parsers, code analysis frameworks, and automated assistants to become outdated quickly [38]. Furthermore, the lack of mature engineering practices for managing the evolution, maintenance, and reuse of quantum software represents an important research gap.

Hardware-related limitations also remain a critical challenge in QSE. The presence of noise, decoherence, gate errors, and limited qubit connectivity in current NISQ devices makes it difficult to distinguish software defects from hardware-induced failures. Consequently, the effectiveness of testing, debugging, and runtime verification techniques is often compromised, particularly for deep quantum circuits where accumulated physical errors reduce the reliability of assertions and statistical oracles [1, 25, 39]. These limitations highlight the need for SE techniques that are more resilient to hardware variability and capable of operating under realistic quantum execution conditions.

The literature also consistently identifies the oracle problem as one of the fundamental barriers to automated quantum software testing. As shown in Figure 3, Oracle/Unobservability remains a recurrent challenge reported across multiple studies. Due to the no-cloning theorem and the collapse of the quantum state after measurement, intermediate quantum states cannot be directly inspected without affecting program execution. Therefore, defining reliable test oracles is considerably more complex than in classical software engineering. To mitigate this issue, several studies have explored alternative validation strategies, including metamorphic testing, differential testing, and equivalence-based approaches, which reduce the dependency on explicit expected outputs [15, 23].

In addition to execution-related challenges, the reviewed studies highlight gaps associated with data availability and interoperability. The scarcity of standardized datasets and repositories containing realistic quantum software defects limits the development and evaluation of data-driven approaches, particularly those based on machine learning and large language models [16]. Similarly, abstraction and interoperability issues indicate that current quantum development ecosystems still lack sufficient mechanisms for integrating different programming languages, frameworks, and execution platforms.

Finally, the reviewed studies reveal that higher-level SE activities remain insufficiently explored. Relatively few approaches address architectural design, reverse engineering, software modernization, and the maintenance of hybrid classical–quantum systems. This finding corroborates the results of RQ1, which showed that current research remains strongly concentrated on implementation and testing, while design, maintenance, and software reuse receive comparatively less attention [10, 28]. Therefore, the evidence summarized in Figure 3 suggests that future research should move beyond isolated technical solutions and investigate comprehensive SE approaches capable of supporting the entire quantum software lifecycle.

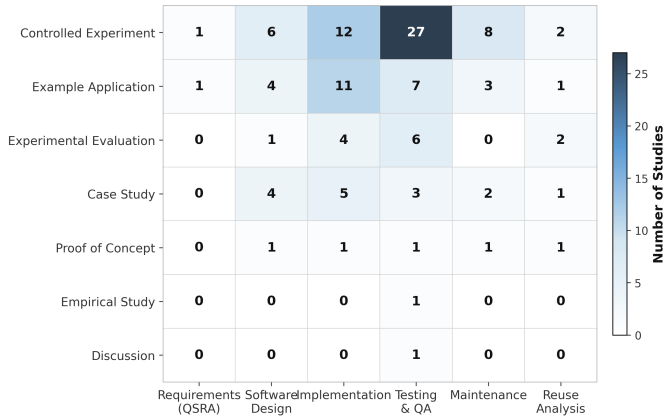
4 Discussion

This SMS provides a comprehensive overview of the current landscape of SE tools for QC. The results indicate that, although the number of proposed tools has grown considerably in recent years, research efforts remain concentrated on a limited subset of the QSDL. In particular, implementation and testing dominate the current landscape, whereas requirements engineering, software design, maintenance, and software reuse remain comparatively underexplored.

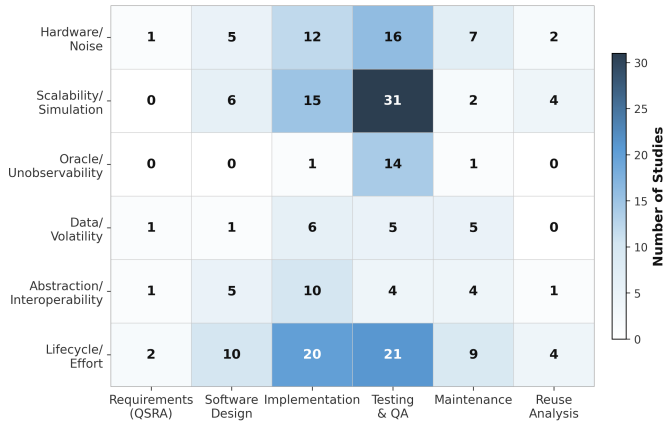
This distribution is also reflected in the maturity of the identified tools, as illustrated in Figure 4, which presents two complementary perspectives. Figure 4a shows that empirical evaluation is largely concentrated in the *Implementation* and *Testing & Quality Assurance* phases, where controlled experiments are the predominant evaluation strategy.

A complementary view is provided by Figure 4b, which correlates the SE limitations addressed by the identified tools with the QSDL phases. The results reveal that most studies focus on overcoming limitations related to *Scalability/Simulation*, *Hardware/Noise*, and *Lifecycle/Effort*, particularly during implementation and testing. This distribution reflects the immediate challenges imposed by current quantum hardware and the complexity of developing, executing, and validating quantum software.

Taken together, these findings indicate that QSE research is currently driven by the practical challenges of implementing and validating quantum programs. While this emphasis is understandable given the constraints of NISQ-era technologies, broader lifecycle support remains limited. Future research should therefore focus on developing integrated toolchains that span the entire QSDL, while also providing stronger empirical evidence through realistic industrial case studies and standardized evaluation benchmarks. Such advances are essential for establishing QSE as a mature discipline capable of supporting reliable, scalable, and maintainable quantum software systems.



(a) Empirical evaluation methods versus QSDL phases.



(b) Software engineering limitations versus QSDL phases.

Figure 4: Heatmaps summarizing the current landscape of Software Engineering tools for QC. (a) Relationship between empirical evaluation methods and the QSDL phases supported by the identified tools. (b) Relationship between the software engineering limitations addressed by the identified tools and the QSDL phases.

5 Threats to Validity

This study presents threats to validity primarily related to the study selection and data extraction processes, as well as to the dynamic nature of Software Engineering for Quantum Computing. Regarding internal validity, a potential threat arises from researchers' judgment during study selection and data extraction. Potential classification errors concerning tool categories, QSDL phases, types of empirical evidence, and reported limitations may introduce biases into the synthesis of the results. To mitigate this risk, a standardized data extraction form was used to systematically record the information required to answer the research questions, contributing to greater consistency and traceability throughout the analysis.

Regarding external validity, the results are subject to the dynamic nature of Software Engineering for Quantum Computing. The findings reflect the state of the art during the analyzed period and may change as new studies, tools, programming languages, and platforms are published. Despite this limitation, the adoption of a systematic protocol and the public availability of the extracted data support the transparency, replication, and future updating of this study.

6 Conclusion

This paper presented a SMS on SE tools for QC, aiming to identify the existing categories of tools, analyze the empirical evidence supporting them, and synthesize the main limitations, challenges, and research gaps reported in the literature. Following a rigorous review protocol, 71 primary studies were selected and analyzed.

The results indicate that the current QSE ecosystem offers a diverse set of tools addressing multiple phases of the QSDL. However, research efforts remain heavily concentrated on software implementation and testing, while comparatively fewer tools support requirements engineering, software design, maintenance, and software reuse. The review also showed that controlled experiments constitute the predominant evaluation strategy, whereas empirical studies involving industrial settings, practitioners, and long-term case studies remain scarce.

The identified research challenges demonstrate that the maturity of QSE is still constrained by both technological and methodological limitations. Scalability issues, the restrictions imposed by NISQ hardware, the oracle problem, limited interoperability between software ecosystems, and the rapid evolution of quantum programming frameworks continue to hinder the development of mature, general-purpose SE solutions. These limitations also help explain the uneven coverage of the QSDL and the predominance of research prototypes over production-ready tools.

Finally, the dataset produced during this review, including the extracted data and study classifications, has been made publicly available to support reproducibility and facilitate future research on QSE.

Artifact Availability

The artifacts supporting this study, including the extracted dataset, data analysis scripts, and generated figures, are publicly available in the companion GitHub repository, available at https://github.com/gems-uff/se4q-review.

Acknowledgements

The authors would like to acknowledge the use of OpenAI's ChatGPT [4] and Google's NotebookLM [5] as AI-assisted tools that supported language refinement, text organization, and literature management throughout the preparation of this study. All analyses, interpretations, methodological decisions, and conclusions presented in this paper remain the sole responsibility of the authors.

The authors also gratefully acknowledge the financial support provided by the National Council for Scientific and Technological Development (CNPq), under grant 309300/2023-1, and the Carlos Chagas Filho Foundation for Research Support of the State of Rio de Janeiro (FAPERJ), under grant E-26/204.145/2024.

[4] https://chatgpt.com/
[5] https://notebooklm.google/

References

[1] Jaime Alvarado-Valiente, Javier Romero-Alvarez, Enrique Moguel, Jose Garcia-Alonso, Macario Polo, Ignacio García-Rodríguez, and Juan M Murillo. 2026. QuMuS: A scalable tool for quantum mutant testing on NISQ hardware. *SoftwareX* 34 (2026), 102651.

[2] Yaiza Aragonés-Soria and Manuel Oriol. 2024. C4q: A chatbot for quantum. In *Proceedings of the 5th ACM/IEEE International Workshop on Quantum Software Engineering*. 29–36.

[3] Victor R Basili, Richard W Selby, and David H Hutchens. 1986. Experimentation in software engineering. *IEEE Transactions on software engineering* 7 (1986), 733–743.

[4] Abdul Basit, Minghao Shao, Muhammad Haider Asif, Nouhaila Innan, Muhammad Kashif, Alberto Marchisio, and Muhammad Shafique. 2025. PennyCoder: Efficient Domain-Specific LLMs for PennyLane-Based Quantum Code Generation. In *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 2. IEEE, 229–234.

[5] Nicolas Dupuis, Luca Buratti, Sanjay Vishwakarma, Aitana Viudes Forrat, David Kremer, Ismael Faro, Ruchir Puri, and Juan Cruz-Benito. 2024. Qiskit code assistant: Training llms for generating quantum computing code. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–4.

[6] Kanishk Dwivedi, Majid Haghparast, and Tommi Mikkonen. 2024. Quantum software engineering and quantum software development lifecycle: a survey. *Cluster Computing* 27, 6 (Sept. 2024), 7127–7145. doi:10.1007/s10586-024-04362-1

[7] Wang Fang and Mingsheng Ying. 2024. Symbolic execution for quantum error correction programs. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 1040–1065.

[8] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 212–219.

[9] Xiaoyu Guo, Shinobu Saito, and Jianjun Zhao. 2025. M2QCode: A Model-Driven Framework for Generating Multi-Platform Quantum Programs. *arXiv preprint arXiv:2510.17110* (2025).

[10] Xiaoyu Guo, Shinobu Saito, and Jianjun Zhao. 2025. QuanUML: Towards a modeling language for model-driven quantum software development. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1344–1349.

[11] Luis Jiménez-Navajas, Ricardo Pérez-Castillo, and Mario Piattini. 2025. Code generation for classical-quantum software systems modeled in UML. *Software and Systems Modeling* 24, 3 (2025), 795–821.

[12] Staffs Keele et al. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report. Technical report, ver. 2.3 ebse technical report. ebse.

[13] Catherine Elizabeth Kendig. 2016. What is proof of concept research and how does it generate epistemic and ethical categories for future scientific practice? *Science and engineering ethics* 22, 3 (2016), 735–753.

[14] Mushahid Khan, Prashant J Nair, and Olivia Di Matteo. 2024. CircInspect: integrating visual circuit analysis, abstraction, and real-time development in quantum debugging. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 1000–1006.

[15] Junjie Luo, Shangzhou Xia, Fuyuan Zhang, and Jianjun Zhao. 2026. QEMI: A Quantum Software Stacks Testing Framework via Equivalence Modulo Inputs. In *International Conference on Fundamental Approaches to Software Engineering*. Springer, 149–169.

[16] Xuan Mao, Zijie Huang, Jianxin Ge, Chao Wang, Wuxu Wang, and Lizhi Cai. 2025. Towards Defect Prediction for Quantum Software. In *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 1–8.

[17] Flavio Melinte-Citea and Mohammad Reza Mousavi. 2026. RuntimeSpeQ: Specifying Runtime Monitors for Quantum Programs. In *Proceedings of the 7th IEEE/ACM International Workshop on Quantum Software Engineering*. 42–49.

[18] N David Mermin. 2007. *Quantum computer science: an introduction*. Cambridge University Press.

[19] Ashley Montanaro. 2016. Quantum algorithms: an overview. *npj Quantum Information* 2, 1 (2016), 1–8.

[20] Hoa T Nguyen, Muhammad Usman, and Rajkumar Buyya. 2024. iQuantum: A toolkit for modeling and simulation of quantum computing environments. *Software: Practice and Experience* 54, 6 (2024), 1141–1171.

[21] Hoa T Nguyen, Muhammad Usman, and Rajkumar Buyya. 2024. Qfaas: A serverless function-as-a-service framework for quantum computing. *Future Generation Computer Systems* 154 (2024), 281–300.

[22] Michael A Nielsen and Isaac L Chuang. 2010. *Quantum computation and quantum information*. Cambridge university press.

[23] Matteo Paltenghi and Michael Pradel. 2023. MorphQ: Metamorphic testing of the Qiskit quantum computing platform. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2413–2424.

[24] Matteo Paltenghi and Michael Pradel. 2024. Analyzing quantum programs with lintq: A static analysis framework for qiskit. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2144–2166.

[25] Tirthak Patel and Devesh Tiwari. 2021. Qraft: Reverse your quantum circuit and know the correct program output. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 443–455.

[26] Michael Quinn Patton. 2014. *Qualitative research & evaluation methods: Integrating theory and practice*. Sage publications.

[27] Ricardo Pérez-Castillo, Luis Jiménez-Navajas, and Mario Piattini. 2021. Modelling quantum circuits with UML. In *2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 7–12.

[28] Ricardo Pérez-Castillo, Luis Jiménez-Navajas, and Mario Piattini. 2022. QRev: migrating quantum code towards hybrid information systems. *Software Quality Journal* 30, 2 (2022), 551–580.

[29] Mario Piattini, Guido Peterssen, Ricardo Pérez-Castillo, Jose Luis Hevia, Manuel A Serrano, Guillermo Hernández, Ignacio García Rodríguez De Guzmán, Claudio Andrés Paradela, Macario Polo, Ezequiel Murina, et al. 2020. The Talavera Manifesto for quantum software engineering and programming.. In *QANSWER*. 1–5.

[30] Mario Piattini, Manuel Serrano, Ricardo Perez-Castillo, Guido Petersen, and Jose Luis Hevia. 2021. Toward a quantum software engineering. *IT Professional* 23, 1 (2021), 62–66.

[31] Gabriel Pontolillo, Mohammad Reza Mousavi, and Marek Grzesiuk. 2025. Qucheck: A property-based testing framework for quantum programs in qiskit. *ACM Transactions on Quantum Computing* (2025).

[32] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. Towards an automated framework for realizing quantum computing solutions. In *2023 IEEE 53rd International Symposium on Multiple-Valued Logic (ISMVL)*. IEEE, 134–140.

[33] Damian Rovara, Lukas Burgholzer, and Robert Wille. 2025. A framework for debugging quantum programs. In *2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, 130–136.

[34] Mary Shaw. 2003. Writing good software engineering research papers. In *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 726–736.

[35] Julian Shen, Joshua Ammermann, Christoph König, and Ina Schaefer. 2025. Quantum Pattern Detection: Accurate State-and Circuit-based Analyses. In *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 9–16.

[36] Peter W Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* 41, 2 (1999), 303–332.

[37] Meghana Sistla, Swarat Chaudhuri, and Thomas Reps. 2023. Symbolic quantum simulation with quasimodo. In *International Conference on Computer Aided Verification*. Springer, 213–225.

[38] Nicholas Stapleton, Sharmin Afrose, Ethan Seefried, Vicente Leyton Ortega, Travis S Humble, and Tirthankar Ghosal. 2026. Quantum-Rag: Retrieval-Augmented Generation Framework for Quantum Circuit Code Generation. In *2026 IEEE 19th Dallas Circuits and Systems Conference (DCAS)*. IEEE, 1–5.

[39] Javier Ibarra Veganzones, Danel Arias Alamo, Alfredo Cuzzocrea, and Pablo Garcıa Bringas. 2025. AssertsQ: A Quantum Assertion Tool for Quantum Software Debugging. (2025).

[40] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2021. Quito: a coverage-guided test generator for quantum programs. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1237–1241.

[41] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2022. QuSBT: Search-based testing of quantum programs. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 173–177.

[42] Benjamin Weder, Johanna Barzen, Martin Beisel, Fabian Bühler, Daniel Georg, Frank Leymann, and Lavinia Stiliadou. 2025. Qunicorn: a middleware for the unified execution across heterogeneous quantum cloud offerings. In *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 17–24.

[43] Lloyd G Williams and Connie U Smith. 1998. Performance evaluation of software architectures. In *Proceedings of the 1st international workshop on Software and performance*. 164–177.

[44] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*. 1–10.

[45] Masaomi Yamaguchi, Nobukazu Yoshioka, and Fuyuki Ishikawa. 2025. Practical Design by Contract Framework for Quantum Applications. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1699–1709.

[46] Robert K Yin. 2009. *Case study research: Design and methods*. Vol. 5. sage.

[47] Carmen Zannier, Grigori Melnik, and Frank Maurer. 2006. On the success of empirical studies in the international conference on software engineering. In *Proceedings of the 28th international conference on Software engineering*. 341–350.

[48] Pengzhan Zhao, Xiongfei Wu, Zhuo Li, and Jianjun Zhao. 2023. Qchecker: Detecting bugs in quantum programs via static analysis. In *2023 IEEE/ACM 4th International Workshop on Quantum Software Engineering (Q-SE)*. IEEE, 50–57.